

Exame de Programação Funcional

3 de Julho de 2012

DCC/FCUP

Versão A

Nome: _____

Nº mecanográfico: _____

- Este exame contém 6 questões em 4 páginas.
- Responda às questões 1, 2, 3 no espaço marcado no enunciado.
- Responda às questões 4, 5 e 6 numa folha de exame separada.
- Duração: 2h + 30m tolerância.

1. (15%) Responda a cada uma das seguintes questões, indicando **apenas** o resultado de cada expressão.

(a) `tail (head [[1,2],[3,4]])` = _____

(b) `map ('mod'2) [1,2,3]` = _____

(c) `length (filter (/='a') "banana")` = _____

(d) `head [n | n<-[1..], n*n>20]` = _____

(e) `[x | x<-"ab", n<-[1,2,3]]` = _____

(f) `foldr (\x y->x:'.':y) "" "ABC"` = _____

(g) Indique um tipo admissível para `([1,2,3], "banana")`

(h) Considere a seguinte definição da função `all` do prelúdio-padrão:

```
all f []      = True
all f (x:xs) = f x && all f xs
```

Indique o tipo mais geral desta definição:

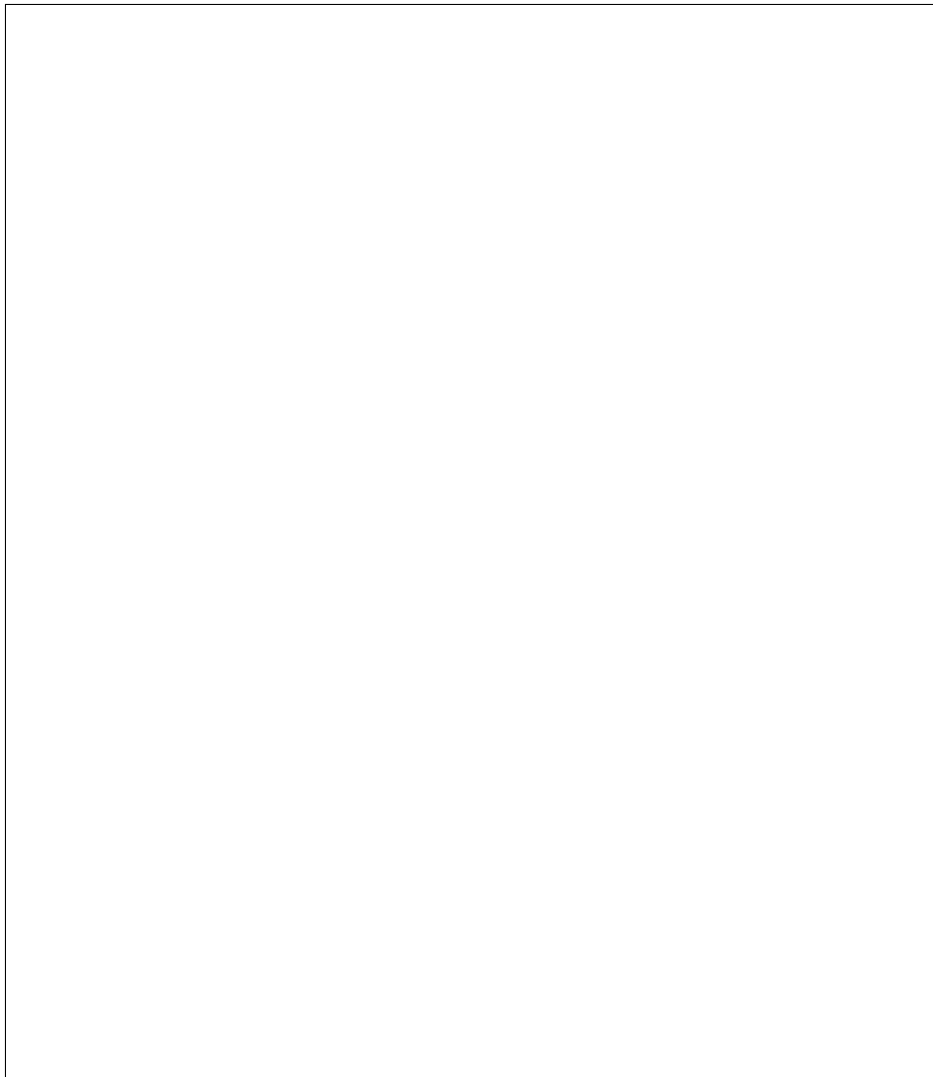
2. (20%) Escreva uma definição recursiva da função

```
prefixo :: String -> String -> Bool
```

que verifica se uma cadeia de caracteres é prefixo de outra.

Exemplos de uso:

```
prefixo "abra" "abra" = True  
prefixo "abra" "abracadabra" = True  
prefixo "braca" "abracadabra" = False  
prefixo "abracadabra" "abra" = False
```



3. (20%) Considere a representação de uma matriz como uma lista de vetores, em que cada vetor é uma lista de coeficientes fracionários.

```
type Mat = [Vec]
type Vec = [Double]
```

- (a) Escreva uma definição da função `somaV :: Vec -> Vec -> Vec` que calcula a soma de dois vetores. Deve verificar se os vetores têm o mesmo número de elementos e dar um erro em caso contrário.
Exemplo: `somaV [1,2,3] [4,5,6] = [5,7,9]`.

- (b) Usando a função `somaV`, escreva uma definição de `somaM :: Mat -> Mat -> Mat` que calcula a soma de duas matrizes (isto é, a matriz com a soma dos vetores em posições correspondentes). Deve verificar se as matrizes têm o mesmo número de vetores e dar um erro caso contrário.
Exemplo: `somaM [[1,2],[3,4]] [[5,6],[7,8]] = [[6,8],[10,12]]`.

Responda às questões 4, 5 e 6 numa folha de exame separada.

4. (15%) Considere a definição em Haskell dum tipo de dados para árvores binárias de pesquisa:

```
data Arv a = Vazia | No a (Arv a) (Arv a)
```

(a) Escreva definições recursivas das funções

```
tamanho :: Arv a -> Int
altura  :: Arv a -> Int
```

que calculam, respetivamente, o *tamanho* (i.e. o número total de nós) e a *altura* (i.e. o maior comprimento de um caminho da raiz até uma folha) duma árvore.

(b) Encontre um **contra-exemplo** para a seguinte afirmação: “se duas árvores têm altura igual, então têm tamanho igual”. Justifique a sua resposta usando as definições da alínea anterior.

5. (15%) Usando indução, prove a seguinte propriedade para quaisquer listas xs e ys .

$$length\ (xs \mathbin{++} ys) = length\ xs + length\ ys$$

Sugestão: use indução sobre xs .

6. (15%) Considere a seguinte definição da função $rld :: [(Int, Char)] \rightarrow String$ que descomprime uma lista de pares com “*run-length encoding*”: cada par (n, x) com $n \geq 0$ representa a sequência de n caracteres x repetidos.

$$rld\ [] = [] \tag{rld.1}$$

$$rld\ ((n, x) : ps) = replicate\ n\ x \mathbin{++} rld\ ps \tag{rld.2}$$

Usando indução sobre listas, mostre que

$$length\ (rld\ ps) = sum\ (map\ fst\ ps)$$

onde ps é uma lista de pares e fst é função projecção na primeira componente do par, i.e. $fst\ (x, y) = x$.

Sugestão: use a propriedade do exercício 5 e o resultado auxiliar seguinte (provado nas aulas): $length\ (replicate\ n\ x) = n$, para $n \geq 0$.